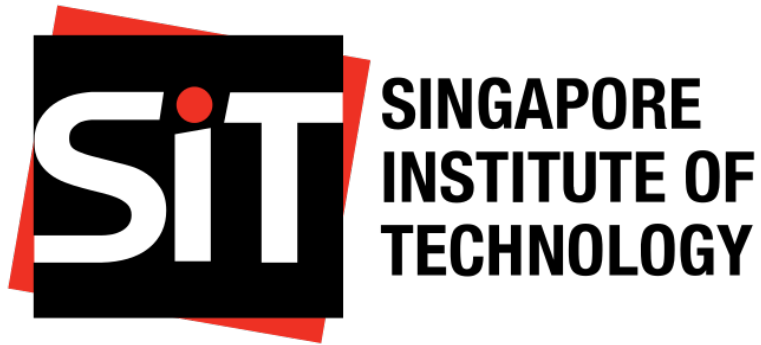




INF2007 - MOBILE APPLICATION DEVELOPMENT

SINGAPORE INSTITUTE OF TECHNOLOGY

INFOCOMM TECHNOLOGY

Final Report: Mobile Application Development Team13

Students:

Subhashini Rajkumar (ID: 2402769)

Lai Yueyin Shyann (ID: 2402456)

Foo Guo Wei (ID: 2402716)

Tan Yuewen Sara (ID: 2402051)

Nathan Teo JiaJun (ID: 2401116)

Date: April 30, 2026

1 Introduction

1.1 Project Overview

This project presents a mobile learning platform that leverages Artificial Intelligence (AI) to support teachers and students in delivering and experiencing personalised learning. Teachers upload course materials as PDFs, and the system automatically generates structured lesson summaries and multiple-choice quizzes via the Gemini API. Students access these materials to reinforce understanding and receive immediate feedback on their performance.

The platform also integrates QR code-based content sharing, real-time progress tracking through analytics dashboards, and a role-based alert system, all backed by Supabase for cloud data management. Designed with a mobile-first approach, it addresses the needs of educators managing multiple modules and students seeking accessible, self-paced study tools.

1.2 Motivation and Background

Modern classrooms are increasingly diverse in learning pace, academic ability, and preferred learning styles. Teachers often face significant time constraints and limited tools to effectively personalise learning. Existing digital platforms may offer content delivery but lack real-time adaptation, teacher-centric AI assistance, or mobile-first designs suited to how students consume content today.

A key challenge identified during the initial proposal was the disconnect between content distribution and assessment. Teachers spend considerable time manually creating study guides and quiz questions from lecture materials, a process that is repetitive and scales poorly across multiple modules. Research in educational psychology consistently demonstrates that active recall through quizzing is among the most effective study strategies, yet producing quality practice questions remains a significant barrier for educators.

Advancements in large language models (LLMs), particularly through the Gemini API, have made it practical to extract meaningful content from uploaded PDFs and transform them into quiz-ready formats with minimal teacher intervention. The growing adoption of mobile-first learning among tertiary students has further reinforced the need for a responsive platform designed for on-the-go study sessions.

The lasting impact of hybrid and self-directed learning models following the COVID-19 pandemic has also shifted student expectations. Students increasingly expect digital tools that allow self-paced study with immediate feedback, while teachers require better visibility into performance to enable timely intervention. These developments shaped our application's direction: from a general adaptive learning concept towards a focused AI-powered content generation and performance monitoring platform.

1.3 Objectives

Our application aims to achieve the following:

- Reduce the time teachers spend on content preparation by automating the generation of reading summaries, key concepts, and quiz questions from uploaded module material using AI.
- Provide students with a structured and accessible way to revise module content and test their understanding through AI-generated quizzes with instant feedback supporting the testing effect established in educational research.
- Deliver real-time analytics dashboards that give teachers visibility into class performance across modules, including class averages active student counts, and weekly performance trends.
- Implement a role-based alert system that identifies students averaging less than 50 percent for 2 or more consecutive weeks, enabling early intervention for at-risk learners.
- Organized all content and assessments by module with folder-based navigation, ensuring that both teachers and students can locate their materials intuitively.
- Support QR code-based content sharing, allowing teachers to distribute lesson materials and quizzes efficiently and students to access them instantly without searching for links.

1.4 Impact Discussion

Societal Impact

- **Inclusive education:** AI-generated study materials are standardised across classes, ensuring every student receives the same structured summaries and practice quizzes regardless of their teacher's experience or preparation time.
- **Digital readiness:** The platform encourages the adoption of intelligent mobile technologies in education, supporting broader digital readiness in academic institutions.

Individual Impact

For teachers, the application transforms a time-intensive manual process into an automated workflow:

- A single PDF upload produces a complete lesson summary and ready-to-publish quiz within minutes.
- The analytics dashboard provides data-driven insights, enabling informed decisions on where to focus support.
- The module-level alert system flags consistently underperforming students, which is particularly valuable when managing large cohorts across multiple modules.

For students, the application promotes consistent study habits and self-awareness:

- Immediate visual feedback on quiz answers supports active recall and review.
- The module-filtered analytics screen allows students to track their own progression over time.
- The alert system serves as an early warning, notifying students of declining performance before it becomes critical.

Validation from User Testing

Preliminary user testing validated several design decisions. AI-generated questions aligned well with source material and covered an appropriate range of difficulty levels. Users found the module-based folder organisation intuitive, and the ability to immediately review incorrect answers was particularly valued. The expandable alert system, which groups failing indicators by module and supports drill-down into specific quizzes or students, was recognised as practical and informative without being overwhelming.

2 Literature Review

2.1 Review of Similar Works

	Features / Functionalities	How it Differs	Improvement / Additional
Duolingo [3]	Gamified adaptive exercises that provides instant feedback to users	Focused more on language learning, catering to learners, not teachers	Teacher-centric AI assistant providing real-time engagement data, supports QR sharing
WING [4] (research project)	Adaptive gamified platform for neurodivergent learners, (usability and multisensory interaction)	Focuses on 1 target group, instead of broad classroom diversity that includes teacher support and analytics	Aims for real classroom deployment that covers both teacher and student workflow, not just adaptive learning paths
Creta Class [1]	AI adaptive and personalized educational mobile app targeting early childhood	Focuses on student learning adaptation on specific subjects for young learners	Supports diverse learning context with teacher-side AI lesson creation and analytics

2.2 Detailed Analysis of Duolingo

Duolingo has more than 50 million daily active users [2], hence we will use Duolingo for our detail analysis. Duolingo has key features such as gamified streaks and adaptive spaced-repetition to attract and maintain high daily active user. However, it had limitation when come into tertiary learning mobile platform. As a 'closed-loop' nature application, although progress monitoring is a provided feature, Duolingo does not provide any tools for curriculum customization. Teachers using their application are not permitted to upload specific module materials or generate assessments that are tailored for their students.

Our application proposes an improvement by a generative model that take an 'open-loop' approach. Integrating dual-role interface, teachers can upload their own materials, which will be used by AI to generate a summary. Using key concepts, AI will generate quizzes using previous generated content. Teachers will be able to amend the quiz as they wish, ensuring the quality. This reduces time spend on repetitive tasks like creating quiz for practice, and focus on high-impact instructional actives. Students benefit from getting more module contents and quizzes to practice without losing it's quality.

2.3 Implementation of Key Mobile Aspects

Mobile Aspect	Reason for Selection	How It Enhances the Application
Mobile AI	To support personalized learning and reduce teachers' workload	Enables adaptive lesson difficulty, AI-generated quizzes, learning materials, visual aids, and personalized feedback based on student performance
Mobile Computing	To allow real-time data processing and analytics on mobile devices	Supports instant performance tracking, engagement monitoring, progress dashboards, and seamless adaptive learning experiences
Mobile Communication	To enable fast and efficient interaction between teachers and students	Allows QR code sharing of lessons and quizzes, push notifications for feedback, and real-time teacher interventions
Mobile Multimedia	To support diverse learning styles and accessibility needs	Provides visual aids, simplified summaries, multimedia-rich lessons, and improved understanding of complex topics

3 System Architecture and Design

3.1 Architecture Overview

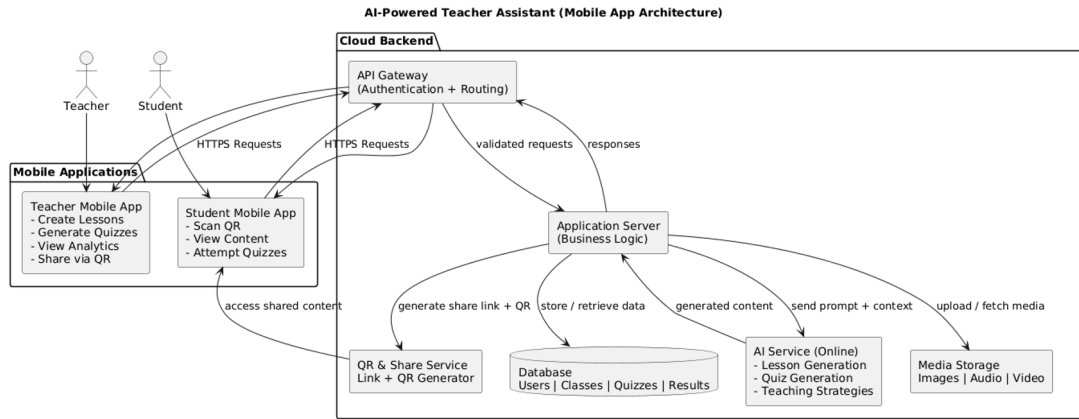


Figure 1: Software Architecture Diagram

Figure 1 illustrates the overall system architecture. The system follows a client-server architecture built with Kotlin and Jetpack Compose on the mobile side, using the MVVM pattern to separate UI, business logic, and data concerns. Supabase provides the cloud backend, offering a managed PostgreSQL database, RESTful API endpoints, file storage, and authentication. The Google Gemini API handles all AI content and quiz generation.

The application requires a minimum API level 29, a camera module for QR code scanning, local storage for preference caching via SharedPreferences, and network connectivity to communicate with the backend. No specialised hardware is required.

The system is organised into four layers:

- **Presentation Layer:** Built with Jetpack Compose, containing all UI screens. Each screen observes its corresponding ViewModel via Kotlin StateFlow, enabling reactive UI updates.
- **ViewModel Layer:** Manages UI state and coordinates between the presentation and data layers. Key ViewModels include LessonViewModel, QuizViewModel, and UserViewModel.
- **Repository Layer:** Abstracts data access logic and communicates with Supabase. Includes ModuleRepository, QuizRepository, and LessonRepository.
- **Service Layer:** Contains the Supabase client configuration, PreferenceManager for local storage, and LessonGenerationService for interfacing with the Gemini API.

3.2 Features

Our application provides two distinct user roles, with features tailored accordingly.

Teacher Features

Feature	Description
AI Content Generation	Upload PDF course materials; the Gemini API automatically extracts text, generates structured summaries, and creates multiple-choice quiz questions. Content is stored in Supabase and made immediately available to students.
Quiz Management	Create, edit, and publish quizzes. Modify AI-generated questions including question text, answer options, correct answers, and difficulty levels. Quizzes remain in draft mode until published.
Analytics Dashboard	Displays class-level performance metrics including class average scores, active vs. total student count, weekly performance trends, and per-quiz statistics. Supports module-level filtering.
Student Performance Monitoring	Flags students whose average score falls below 50% for two or more consecutive weeks. Alerts group at-risk students by module, with an expandable view showing average score, number of failing weeks, and attempt count.
QR Code Access	Generates QR codes for lessons and quizzes, enabling instant student access via scanning.
Content Organisation	Lessons and quizzes are organised into module folders, supporting structured navigation across courses.

Table 1: Teacher Features

Student Features

Feature	Description
Quiz Interaction	Answer quiz questions with a progress indicator. Receive instant feedback upon submission: correct answers are highlighted green and incorrect answers red. Final score is displayed clearly at the end.
Quiz Review	Revisit completed quizzes to view selected answers alongside correct answers, using the same visual highlighting as the submission screen.
Learning Content Access	View AI-generated lesson summaries and detailed content. Content is structured with a highlighted summary card followed by full detailed material.
Personal Analytics	View personal performance metrics including overall average score, best score, weekly trend chart, and a list of quiz results. Supports module-level filtering for targeted analysis.
Performance Alerts	Identifies modules where the student's average falls below 50% for two or more consecutive weeks. Expandable details show individual quizzes, scores, and attempt counts.
QR Code Access	Scan QR codes to access lessons and quizzes instantly, with no manual navigation required.

Table 2: Student Features

3.3 Design Details

Front-End Design

The front-end uses Jetpack Compose for a declarative, reactive UI architecture:

- Screens are implemented as composable functions, enabling automatic UI updates on state changes.
- A single `NavController` manages navigation, supporting route-based transitions and parameter passing (e.g. quiz IDs, scores).
- Role-based navigation groups lessons, quizzes, and alerts by module using expandable components.
- UI state is persisted using `rememberSaveable`, ensuring consistency across configuration changes.
- Local preferences such as analytics filters are stored via `SharedPreferences`, avoiding unnecessary backend calls.

Backend and Database Design

The backend uses Supabase for scalable, structured data management:

- A PostgreSQL database with auto-generated RESTful APIs simplifies backend development.
- Core entities include *Profiles* (authentication and roles), *Modules* (course structures), and junction tables linking users to modules.
- Lessons and quizzes support both static and AI-generated content. Quiz-related tables separate metadata, questions, attempts, and answers for flexibility.
- Fine-grained answer tracking enables detailed review and analytics functionality, while a controlled data access chain ensures students only retrieve content from enrolled modules.

3.4 AI Integration

The AI pipeline operates through `LessonGenerationService`, which interfaces with the Gemini API. When a teacher uploads a PDF, the file is stored in Supabase Storage and a public URL is generated. The extracted text is sent to the Gemini API with a structured prompt requesting a lesson summary and detailed content. A second API call generates multiple-choice questions based on the same text, each with four options and a correct answer index. All generated content is stored in the relevant Supabase tables and linked to a new quiz in draft status. Teachers can review and edit the AI-generated content before publishing, maintaining pedagogical control over the materials.

3.5 Data Flow

Content Creation Flow

A teacher selects a PDF from their device, which is transmitted via HTTPS to Supabase Storage. The extracted text is sent to the Gemini API, which returns structured JSON containing the lesson summary, reading content, and quiz questions. This data is parsed by `LessonGenerationService` and inserted into the respective Supabase tables. The teacher's UI updates reactively via `StateFlow` observers in the `ViewModel` layer.

Quiz Attempt Flow

When a student submits a quiz, the score is calculated locally, then a `quiz_attempts` record and individual `quiz_answers` records are written to Supabase simultaneously. On retake, previous answers are replaced to ensure the review screen always reflects the most recent attempt.

Analytics and Alert Flow

The analytics screen fetches lessons, quizzes, and attempts from Supabase on load. Filtering, grouping, and computations such as class averages, weekly trends, and low-scorer identification are performed locally on the device. The alert system groups attempts by module and counts consecutive weeks where the average falls below 50%, reducing backend complexity by avoiding custom server-side aggregation.

QR Code Flow

Lesson and quiz identifiers are encoded into QR codes generated on the teacher's device. When a

student scans a code using the device camera, the identifier is extracted and used to navigate directly to the corresponding lesson or quiz screen, bypassing standard folder navigation.

4 Implementation and Development

Integration of Mobile Aspects

Mobile Aspect	Implement	How It Enhances the Application
Mobile Sensors (Camera)	Device camera integrated via Android CameraX API for QR code scanning. Teachers generate QR codes containing lesson or quiz identifiers; students scan them to navigate directly to content.	Eliminates manual searching through module folders and provides instant content access in live classroom settings, leveraging existing device hardware without additional equipment.
Mobile Communication (HTTPS)	All client-server communication occurs over HTTPS through Supabase's PostgREST API. Network calls execute asynchronously using Kotlin Coroutines, with a custom <code>safeSupabaseCall</code> wrapper for error handling.	Enables real-time data synchronisation between teacher uploads and student access. Asynchronous execution prevents UI freezing, and graceful error handling keeps the application stable on poor network connections.
Mobile Computing (On-Device Processing)	Analytics computations: includes class averages, student de-duplication, weekly trend analysis, consecutive-week failure detection, and module filtering performed locally on the device.	Reduces backend complexity by eliminating custom server-side aggregation, allowing analytics to function with standard Supabase queries and providing responsive UI updates without waiting for server processing.
Mobile AI (Gemini API)	The Google Gemini API processes uploaded PDF text to generate lesson summaries, detailed reading content, and multiple-choice quiz questions. Content is returned as structured JSON, parsed, and stored in Supabase.	Automates the most time-consuming part of the teaching workflow. A single PDF upload produces a complete lesson and quiz within minutes, while teachers retain editorial control by reviewing AI output before publishing.

Table 3: Integration of Mobile Aspects

Technologies Used

Category	Technology	Purpose
Front-End	Kotlin, Jetpack Compose	Declarative UI framework with reactive state management via StateFlow and mutableStateOf.
Navigation	Jetpack Navigation Component	Single-activity architecture with route-based navigation and parameter passing.
Back-End	Supabase (PostgreSQL + PostgREST)	Managed database with auto-generated RESTful APIs, PDF file storage, and row-level security.
AI Service	Google Gemini API	Processes course materials to generate reading content and quiz questions from structured prompts.
Authentication	BCrypt	Password hashing stored in the Profiles table, with an in-memory User Session singleton for session management.
Local Storage	Android SharedPreferences	Persists user preferences such as analytics module filters across app restarts via a custom PreferenceManager.
Version Control	Git, GitHub	Source code management using a branching strategy with main, dev, and feature branches.
CI/CD	GitHub Actions (YAML)	Automated pipeline for linting, unit testing, and debug/release APK builds including instrumented testing on emulator.
Unit Testing	JUnit 4	Validates business logic including score calculations, de-duplication, weekly trends, alert detection, and answer mapping.
Integration Testing	Jetpack Compose Testing	Verifies screen rendering, role-based UI display, navigation callbacks, and network-connected screen behaviour.

Table 4: Technologies Used

Enhancements Made During Development

Issue	Root Cause	Enhancement Applied
App freezing (ANR) on screen load	Multiple sequential Supabase calls during composition, including looping through modules to count students.	Consolidated into single batch queries using <code>getStudentCountForTeacher</code> . Removed redundant <code>init</code> blocks in ViewModels that triggered duplicate data fetches.
Quiz review only showed correct answers, not student selections	Only aggregate scores were stored in <code>quiz_attempts</code> ; individual answer selections were not recorded.	Added <code>quiz_answers</code> table storing each question's selected answer, correct answer, and correctness flag. Review screen now highlights the student's actual choices in green or red.
Alerts persisted after student performance improved	Alert system counted all historical weeks below 50%, not just recent consecutive ones.	Refined to count only consecutive recent failing weeks by iterating backwards from the most recent week and stopping at the first passing week. Alerts now clear automatically on improvement.
Student alerts flagged individual quizzes instead of module performance	Alert logic operated per quiz rather than per module.	Restructured to aggregate all quiz attempts within a module. Expandable cards allow drill-down from module level to individual quizzes.
Home screen quiz card did not auto-expand the correct folder	Quizzes screen always opened with all folders collapsed.	Home screen now passes the module code as a navigation parameter; the Quizzes screen uses it to auto-expand the corresponding folder.
Analytics module filter reset on every visit	Used <code>remember</code> , which resets on navigation.	Changed to <code>rememberSaveable</code> with <code>PreferenceManager</code> persistence via <code>SharedPreferences</code> , surviving both navigation and app restarts.

Table 5: Enhancements Made During Development

User Interface

Figure	Screen	Description
Figure 10	Login Screen	Provides input fields for user authentication and access to the app.

Continued on next page

(Continued)

Figure	Screen	Description
Figure 11	Teacher Home Screen	Displays live statistics (lesson count, quiz count, total attempts, activity status), quick action buttons for uploading lessons and viewing quizzes, and a recent quizzes list with participation ratios and average scores.
Figure 12	Student Home Screen	Features a prominent Scan QR Code card, a Browse Lessons button, and a recent quiz history list with score badges colour-coded by pass or fail status.
Figure 13	Lesson Content Screen	Displays the educational materials and lesson details for students.
Figure 14	Student Scan QR Screen	Interface for students to scan QR codes for attendance or lesson access.
Figure 15	Teacher Create Lesson Screen	Allows teachers to input and save new lesson information.
Figure 16	Teacher Create Quiz Screen	Interface for teachers to generate and configure new quiz modules.
Figure 17	Teacher Edit Quiz Screen	Enables teachers to modify existing quiz questions and settings.
Figure 18	Teacher View QR Screen	Displays the generated QR code for teachers to present to the class.
Figure 19	Teacher Analytics Screen	Includes a module filter dropdown, class overview card (average score and active student count), weekly performance trend chart, list of students below 50%, and per-quiz breakdown.
Figure 20	Student Analytics Screen	Shows personal average score, best score, weekly progress chart, and a scrollable quiz results list with percentage badges.
Figure 21	Quiz Taking Screen	Displays four selectable answer cards per question with a progress bar and submit button. After submission, correct answers are highlighted green and incorrect answers red, with a score summary card.
Figure 22	Quiz Review Screen	Loads the student's saved answers from the database and displays green and red highlighting matching the submission screen, allowing students to see exactly which answers were incorrect.

Continued on next page

(Continued)

Figure	Screen	Description
Figure 23	Teacher Alert Screen	Shows a summary banner with at-risk student count. Expandable module cards display failure counts; tapping reveals individual students with their average score, failing week count, and attempt count.
Figure 24	Student Alert Screen	Lists flagged modules where performance has fallen below 50% for two or more consecutive weeks. Tapping a module expands it to show specific quizzes with individual scores and attempt counts.
Figure 25	Settings Screen	Displays a live alert count and a logout button.

5 Project Management and Planning

5.1 Agile Development Process

Our team created an overview (Figure 8) of all the points we had to tackle before adopting an Agile Scrum methodology with a four-sprint development cycle, spanning approximately 25 days from 7 March to 1 April 2026. Each sprint was planned for approximately 8 days, with clearly defined goals, feature assignments, and acceptance criteria established at the beginning of each sprint.

Sprint planning sessions were conducted at the beginning of each sprint, where the team reviewed the product backlog, identified the highest priority items for the next sprint, and split the work equally between each individual teammate. Each feature was scoped with acceptance criteria and estimated story points to ensure manageable workloads within the sprint.

We maintained a daily communication through team messaging to resolve any issue encountered and handover of work, as some of the function cannot continue without the previous. For instance, in sprint 2 without backend we cannot wire the front-end and test for it functionality for lesson and quiz.

Sprint reviews were conducted at the end of each sprint where completed features were demonstrated, merged into the 'dev' branch, and tested for integration issue. A dedicated team member performed integration testing on the dev branch before merging into main, ensuring that cross-feature bugs were caught before reaching the stable branch.

The team use GitHub project for issue tracking and project management. Issues were created based on acceptance criteria from the product backlog and organized into parent issue corresponding to each feature. We use a Gantt chart (Figure 9) to help us with visualization of the sprint timeline and feature dependencies across iterations

5.2 User Story

User stories were derived from the core problem the application addresses: teachers spend significant time creating study materials and quizzes manually, while students lack accessible tools for structured revision with immediate feedback. Our team identified two primary user personas, teachers and students, and mapped their needs to specific user stories through discussions about classroom workflows and learning behaviour.

Each user story follows the standard format of "As a [role] , I want [action], so that [benefit]" and was selected based on three criteria: alignment with the project's AI-powered content generation focus, feasibility within the three-split timeline, and measurable acceptance criteria that could be verified through testing,

Teacher User Stories

USID	User Story
T1	As a teacher, I want the AI to adapt lesson difficulty based on past student performance, so that mixed-ability classrooms are better supported.
T2	As a teacher, I want to modify AI-generated content directly on my phone, so that I can make quick changes anytime, anywhere.
T3	As a teacher, I want to see how each students are doing in real time, so that I can intervene early.
T4	As a teacher, I want to send personalized feedback suggestions generated by AI, so that students receive timely and meaningful guidance.
T5	As a teacher, I want to share learning materials via QR codes, so that students can quickly access content using their phones.

Student User Stories

USID	User Story
S1	As a student, I want lessons to adapt to my learning pace and performance, so that I am neither bored nor overwhelmed.
S2	As a student, I want to track my progress visually, so that I understand how well I am doing over time.
S3	As a student, I want simplified summaries, so that complex topics are easier to understand.
S4	As a student, I want to scan a QR code to instantly access lesson materials or quizzes, so that I do not waste time searching for links.
S5	As a student, I want instant feedback after completing quizzes, so that I can immediately learn from my mistakes.

5.3 Product Backlog

Priority	Feature Item	User Story Summary	Acceptance Criteria	Estimated Story Points
1	Adaptive Learning Engine	AI adjusts difficulty/pace based on student performance	- After viewing the progress of students, Teachers must be able to manually decide the difficulty suggestion. - The transition between difficulty levels must be seamless for the student.	13
2	QR Access System	Scan QR codes for instant access to lessons and quizzes	- Teachers can click a "QR code" button on any lesson or quiz page. - The QR code must be displayable in full screen. - QR codes must remain valid while following proper permission based logic	3
3	Instant Quiz Feedback	Students receive immediate corrections and AI-generated guidance after quizzes.	- Immediately after submitting a quiz, the student must see a summary of correct vs. incorrect answers. - For every incorrect answer, an AI-generated explanation (max 50 words) must explain why the chosen answer was wrong. - The feedback must allow students to view previous progress on quiz again with no restriction for study material.	5
4	AI Content Mobile Editor	Mobile-responsive UI for teachers to edit AI-generated lesson content on the go.	- The editor UI must be optimized for mobile screens (touch-friendly buttons, no horizontal scrolling). - Teachers must be able to tap any AI-generated text block to trigger an inline text cursor. - A "Publish" button must be present to push changes to the student view instantly.	8

Item Number	Feature Item	User Story Summary	Acceptance Criteria	Estimated Story Points
5	Engagement Dashboard	Real-time monitoring of student activity and performance alerts for teachers	- The dashboard must display a list of all students currently logged into the lesson. - A "Performance" status (red indicator) must trigger if a student has not been doing well in their quizzes. - Teachers must be able to click a student's name to view the student's status on their studies.	8
6	Visual Progress Tracker	Dashboard for students to view their learning journey and performance trends	- Students must have access to a "Analytics" tab featuring a line graph of scores over time. - The tracker must display "Performance" on quizzes the student participated. - An alert indicator must show based on the proficiency level of how they are faring.	5
7	AI Study Aids	Automatic generation of simplified summaries for complex topics	- For any lesson, the system must generate a "Quick Summary" bulleted list. - The system must generate at least one relevant metaphor for complex scientific or mathematical concepts.	8

5.4 Sprint Backlog Summary

Sprint	Duration	Goal	Key Deliverable
1	Before 7 March	Foundation (authentication, basic UI, database setup)	Login / Auth screens, Home / Dashboard UI, basic backend APIs, database setup, Lessons and Quizzes screen UI (hard-coded)
2	7 March to 15 March	Wire up screens to real data, AI quiz generation end-to-end	Lessons and Quizzes screens connected to Supabase, Gemini API quiz generation, quiz editor, quiz database operations, integration testing

Sprint	Duration	Goal	Key Deliverable
3	19 March to 23 March	Teacher home screen, activity tracking , quiz difficulty, QR sharing	Teacher and student dashboards, quiz attempts data layer, difficulty levels for quiz generation, quiz editor wired to Supabase, QR code sharing
4	24 March to 28 March	Student-side features, analytics, alerts, documentation	QR scanner and lesson viewer, quiz taking with instant feedback, quiz history, student analytics, teacher and student alert system, video demo, final report

5.5 Individual Contributions

Team Member	Sprint 1	Sprint 2	Sprint 3	Sprint 4
Subhashini	Basic backend APIs	AI quiz generation backend (Gemini API integration, structured JSON response, mock JSON on Day 1)	Quiz difficulty levels (easy/medium/hard parameter in AI prompt, drop down UI. persistence)	Quiz taking screen, result screen with AI explanation, quiz review screen
Shyann	Document upload feature	Update Lesson page UI and Quiz editor UI (edit, recorder, delete questions, save)	Home screen (teacher and student dashboards with live stats, role-based rendering)	Student QR scanner with CameraX and ML kit, lesson viewer, deep link navigation
Guo Wei	Login/Auth screen, Home/Dashboard UI, Quizzes screen UI	Wire hardcoded Lessons and Quizzes to Supabase, Quiz generation UI	Activity tracking data layer (quiz_attempts table, QuizAttemptRow model, saveAttempt, fetchAttempt)	Transitioned to documentation: architecture overview, Supabase schema documentation, video demo
Sara	Database setup	Quiz database operations (save draft, publish, link to lesson, fetch by lesson/teacher)	QR code generation (ZXing library.deep link encoding, QR modal on quiz and lesson cards)	Transitioned to documentation: sprint retrospectives, CI/CD writeup

Team Member	Sprint 1	Sprint 2	Sprint 3	Sprint 4
Nathan	Basic test cases	Sprint 2 integration testing (generate to edit to save to publish flow, cross branch bug fixes, and merge to dev)	Quiz editor Supabase wiring (replace mock data with real fetch/save, loading and error states)	Student quiz history, teacher dashboard live stats from quiz.attempts, documentation support

5.6 Branching Model

Our team followed a structured Git branching model designed to maintain a stable main branch at all times while allowing parallel feature development.

Branch Structure

Branch	Purpose	Who Merges
main	Stable, production-ready code. Only receives merges from dev after integration testing.	Nathan (after testing dev)
dev	Integration branch. All feature branches merge here at the end of each sprint.	Everyone via pull request
feature/*	Individual feature branches created from dev. Each feature has its own own isolated branch	Feature owner via pull Request to dev

Branch Structure

Our team operated under the following rules throughout the project. Each new feature receives its own branch with a descriptive name following the naming convention in this format `feature/<feature-name>`. If a team member is continuing or extending an existing feature from a previous sprint, they reuse the same branch rather than creating a new one. All feature branches target dev, never main. No team member should pushes directly to main branch under any circumstance.

Merge Workflow

When a feature is complete, our team member opens a pull request from their feature branch into dev. The GitHub Actions CI/CD pipeline automatically triggers on the pull request, running lint analysis and all unit tests. If the CI check fails, the merge is blocked until the issue resolved. This automated gate keeping ensures that no broken code enters the dev branch. Once all feature branches for a sprint are merged into dev, the designated integration tester verifies cross-feature compatibility on the dev branch, resolves any integration bugs, and then merges dev into main.

Branch Life cycle Example

A typical feature life cycle followed this example sequence: the developer creates `feature/student-alerts`

from the latest dev, develops and commits locally with descriptive messages, pushes to origin and opens a pull request targeting dev, CI runs lint and unit tests automatically, the team reviews the pull request and requests changes if needed, once approved and CI passes the branch is merged into dev, and at the end of the sprint dev is merged into main after integration testing passes.

6 Testing and Evaluation

6.1 Testing Methodology

The testing strategy follows a two-tier approach consisting of unit tests and integration tests, complemented by a CI/CD pipeline that automates test execution on every code push.

Unit tests validate individual functions and business logic in isolation without requiring an Android device or emulator. They run on the local JVM and cover core computations such as score calculations, data filtering, student de-duplication, weekly trend analysis, and alert detection logic. Integration tests verify that UI screens render correctly, display appropriate content based on user role, and respond to user interactions as expected. These run on an Android emulator or physical device using the Jetpack Compose Testing framework.

All tests are executed automatically through a GitHub Actions CI/CD pipeline triggered on every push to the main and dev branches and on pull requests targeting those branches. It runs five sequential jobs: lint analysis, unit test execution, debug APK build, instrumented test execution on an emulator, and release APK build, ensuring no code reaches the main branch without passing all automated checks.

6.2 Unit Tests

Unit tests are located in the `app/src/test/java/com/example/mobileappproject/` directory and execute without an Android device. A total of 134 unit tests are organised across 15 test files, each targeting a specific area of business logic. Refer to Table 12: Unit Test table in the appendix for more information for unit testing and Table 13: Unit Testing Expected Result table for unit test expected result respectively.

6.3 Integration Tests

Integration tests are located in the `app/src/androidTest/java/com/example/mobileappproject/` directory and require an Android emulator or device to execute. A total of 40 integration tests are organised across 9 test files. Refer to Table 14: Integration Tests table in the appendix for more information for integration tests and Table 15: Test Expected Result for integration test expected result,

Integration tests that depend on live Supabase data use `waitUntil(10000)` to allow up to 10 seconds for network responses before asserting. `PreferenceManager.init(context)` is called in the test setup to initialise `SharedPreferences` in the test environment.

6.4 CI/CD Pipeline

The automated testing pipeline is defined in `.github/workflows/main.yml` and executes the following jobs in sequence

Job	Runner	Command	Purpose
Lint	ubuntu-latest	<code>./gradlew lint</code>	Static code analysis for potential bugs, unused resources, and accessibility issues
Unit Test	ubuntu-latest	<code>./gradlew test</code>	Executes all 134 unit tests without requiring an Android device
Build Debug APK	ubuntu-latest	<code>./gradlew assembleDebug</code>	Verifies the project compiles successfully into a distributable APK
Instrumented	ubuntu-latest with android Emulator	<code>./gradlew connectedDebugAndroidTest</code>	Executes all 40 integration tests on an API 29 emulator
Build Release APK	ubuntu-latest	<code>./gradlew assembleRelease</code>	Produces a release-ready APK only on pushes to main or dev

The pipeline uses concurrency groups to cancel in-progress runs when new commits are pushed, preventing resource waste. Test result reports and APK artifacts are uploaded and retained for seven days for debugging purposes. Environment secrets including the Gemini API key are injected through GitHub Secrets and written to `local.properties` during the building process.

6.5 Experimental Evaluation

The application was evaluated against its six stated objectives through functional testing with test user accounts and automated test validation.

Objective	Metric	Method	Result
Reduce teacher content preparation time	Time from PDF upload to published quiz	End-to-end measurement with a sample lecture PDF	A single upload produces a complete lesson summary and 5-question quiz in approximately 2 minutes, compared to an estimated 30–45 minutes manually.

Provide structured quiz practice with instant feedback	Correctness of feedback display	Integration tests and manual testing	Submission immediately highlights correct answers green and incorrect answers red. Review screen correctly loads saved answers. 6 unit tests validate answer mapping logic.
Deliver real-time analytics	Accuracy of displayed metrics	Analytics output compared against raw database values via SQL queries	Class averages, unique student counts, and weekly trends matched manual calculations from <code>quiz_attempts</code> data. 5 unit tests validate filtering and computation logic.
Identify at-risk students through alerts	Alert accuracy and clearing behaviour	Controlled quiz attempt data tested across multiple weeks	Students averaging below 50% for 2 or more consecutive weeks are correctly flagged. Alerts clear automatically when the student passes the most recent week. 14 unit tests validate detection logic.
Organise content by module	Correct grouping and navigation	Integration tests and manual testing across multiple modules	Lessons, quizzes, and alerts correctly group by module code. Folder expansion state persists via <code>rememberSaveable</code> . 6 unit tests validate grouping logic.
Support QR code content sharing	Successful scan-to-content navigation	Manual testing with generated QR codes	Scanning a QR code navigates directly to the corresponding lesson or quiz screen without manual folder navigation.

Table 11: Experimental Evaluation

The overall test suite comprises 134 unit tests and 40 integration tests (174 total), all integrated into the CI/CD pipeline. Unit tests execute in under 5 seconds on the local JVM; integration tests complete within a few minutes on the emulator including network wait time.

7 Conclusion

7.1 Achievements and Innovations

The application successfully demonstrates that AI-powered content generation can be meaningfully integrated into a mobile learning platform to address real inefficiencies in the teaching and learning workflow. All six stated objectives were achieved, delivering a functional Android application that serves both teachers and students through a single role-based interface.

The most significant innovation is the end-to-end automation of the content preparation pipeline. A teacher uploads a single PDF and receives a structured lesson summary, detailed reading content, and a complete set of multiple-choice quiz questions within minutes. A process that would otherwise take 30–45 minutes of manual effort per lesson. By integrating the Gemini API directly into the mobile workflow, this capability is accessible from anywhere at any time, consistent with the mobile-first philosophy that guided the project from the outset.

A second key achievement is the analytics and alert system, which operates entirely through on-device computation. Rather than requiring custom backend aggregation, the application fetches raw data from Supabase and performs all statistical analysis locally. These includes class averages, weekly trend computation, student de-duplication, and consecutive-week failure detection. This kept the backend simple while delivering analytics capabilities that would typically require a dedicated reporting service. The alert logic, which clears automatically when a student's performance improves, ensures teachers see actionable and current information rather than historical noise.

The quiz answer persistence feature, added during development in response to testing observations, meaningfully improved the learning experience. By storing each student's individual answer selections and displaying them with the same green and red visual feedback used during submission, the review screen becomes a genuine learning tool rather than a simple score display.

From a software engineering perspective, the project achieved comprehensive automated test coverage with 134 unit tests and 40 integration tests integrated into a GitHub Actions CI/CD pipeline. This ensured every code change was validated against business logic correctness and UI rendering accuracy before reaching the main branch. A practice that proved particularly valuable when refactoring the analytics filtering and alert detection logic.

7.2 Future Directions

Several areas present opportunities for further development.

Adaptive difficulty generation. The current AI pipeline generates content in a single pass without adapting to student performance. A natural evolution would be to analyse class-wide quiz results and automatically generate follow-up quizzes focused on topics where students performed poorly, closing the loop between assessment data and content creation.

AI-driven intervention suggestions. The alert system currently identifies at-risk students but provides no recommendations. Integrating AI-generated suggestions, such as recommending specific lessons for review or generating personalised study plans based on a student's incorrect answer patterns, extends the platform from detection to remediation.

Push notifications. Notifying students when new lessons or quizzes are published, when their alert status changes, or when a teacher provides feedback would increase engagement and ensure timely access to new content without requiring students to manually check the application.

Multimodal content support. The application currently processes text-based PDFs but does not handle images, diagrams, or mathematical notation. Extending the AI pipeline to support multimodal content through vision-capable language models would significantly broaden the range of materials that can be processed.

Collaborative features. Discussion threads on lessons, peer study groups within modules, and teacher-to-student messaging would transform the platform from a content delivery tool into a more complete learning environment, building on research showing that social learning features improve engagement and retention when combined with individual practice.

Authentication improvements. The current system uses application-level BCrypt password hashing. Migrating to Supabase's built-in authentication with OAuth support (e.g. Google Sign-In) would improve security, reduce account creation friction, and align with authentication patterns students are already familiar with.

These directions build on the current foundation, extending the application from an AI-powered content generation tool into a comprehensive adaptive learning ecosystem.

GenAI Declaration

Tools and Resources Used

The following generative AI tools and external resources were used in the development of this project:

- **ChatGPT** (OpenAI) – Used for coding assistance during development
- **Gemini** (Google) – Used for coding assistance during development, and integrated via the **Gemini API** as a core application feature
- **Claude** (Anthropic) – Used for coding assistance and writing/editing of project documentation

Use of AI Output

- **Coding:** AI tools (ChatGPT, Gemini, Claude) were consulted to assist with writing, debugging, and understanding code throughout the development process.
- **In-App Feature:** The Gemini API was integrated directly into the application to generate automated summaries for educational content presented to users.
- **Documentation:** AI tools were used to assist in drafting and editing written sections of this report.

External Support Acknowledged

All AI-generated outputs were reviewed and adapted by the team. The final implementation, design decisions, and written content remain the work of the project team.

Appendix

Appendix A: UI Prototype

Login Page First interface user encounter when launching our application. Users will be prompted to enter email and password or sign up if he or she does not have an account yet.

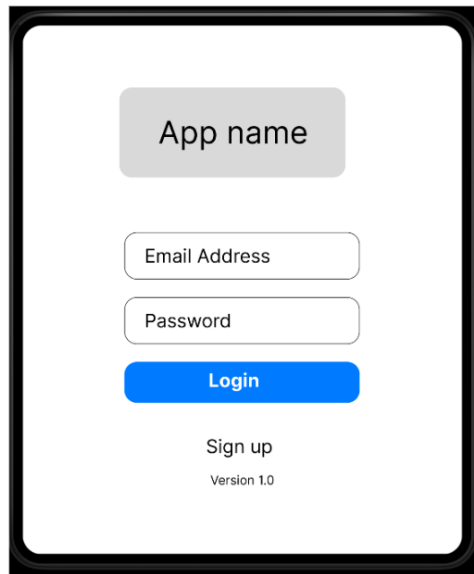


Figure 2: Login Page

Home Page The home page is a scrollable page where the user can view different information at one glance such as average attendance, upcoming lessons and recent activities.

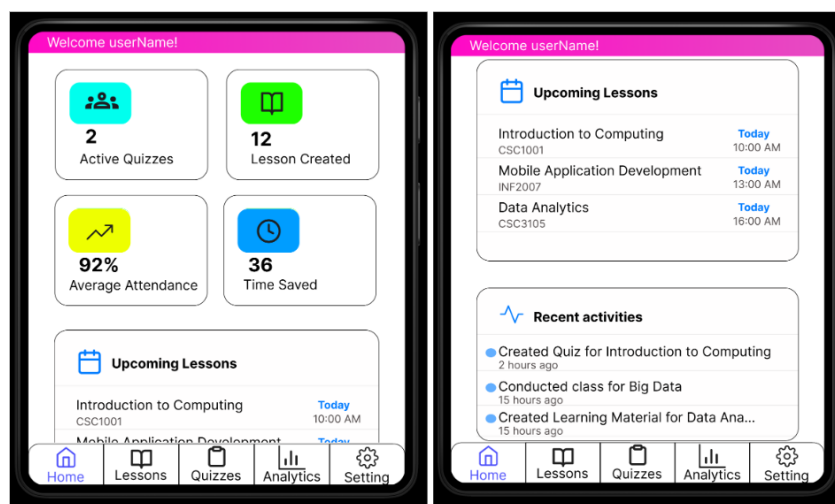


Figure 3: Home Page

Lesson Page Users can upload documents, images or videos and use the AI tools to help them create their quizzes for the students to practice. Users can also edit the upcoming lesson and share their QR code for the students to join.

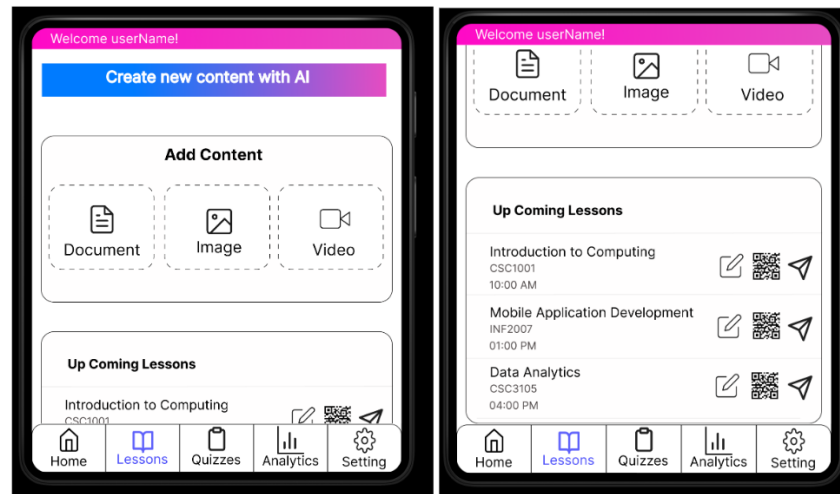


Figure 4: Lesson Page

Quizzes Page After the student conducts the quizzes, the average score for that particular quiz, number of students completed the quiz and total number of students in the module can be seen.

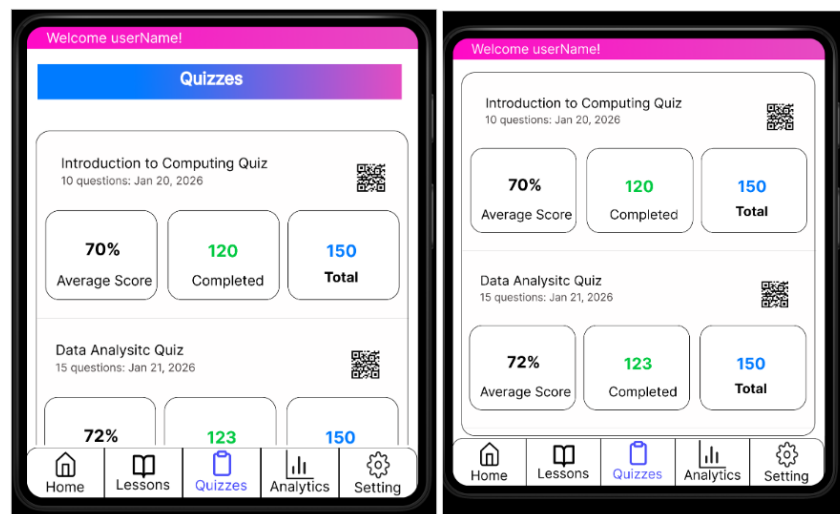


Figure 5: Quizzes Page

Analysis page In the analytics page, it analyzes average increase or decrease in quiz results and the number of active students attending the practice quiz. There are also charts to help with visualization. Users can also see the number of students with names that failed a certain percentage, for example 40%. Based on the information users can help to tailor practice quiz

that helps to address and improve students weaknesses.

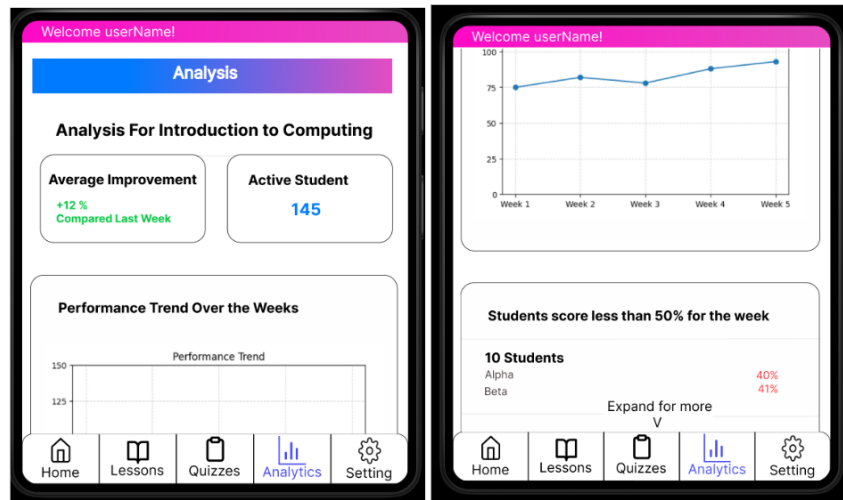


Figure 6: Analysis Page

Setting page In the settings page the user can set alerts to see important messages such as number of students that failed the practice quiz for how many attempts. They can also log out if they want to.

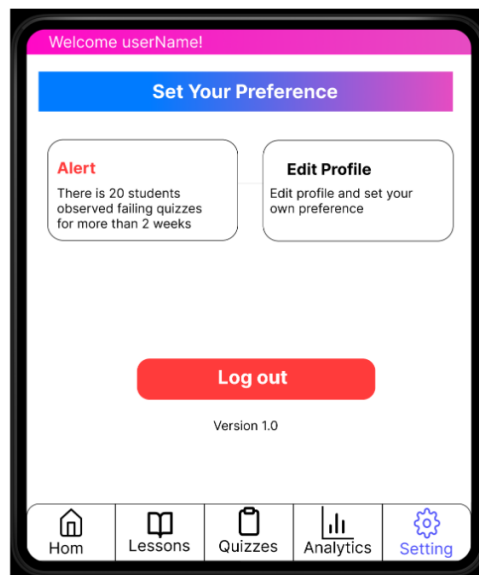


Figure 7: Setting Page

Appendix B: GitHub Projects and Issue Management

Title	Status	Iteration	Category	Parent issue	+
1 Adaptive Learning Engine #1	Todo		Adaptive Learning Engine		
2 QR Access System #2	Todo		QR Access System		
3 Instant Quiz Feedback #9	Todo		Instant Quiz Feedback		
4 AI Content Mobile Editor #10	Todo		AI Content Mobile Editor		
5 Engagement Dashboard #11	Todo		Engagement Dashboard		
6 AI Personalized Feedback #23	Todo		AI Personalised Feedback		
7 Visual Progress Tracker #28	Todo		Visual Progress Tracker		
8 AI Study Aids #32	Todo		AI Study Aids		
9 Dynamic difficulty level #3	Todo	Iteration		Adaptive Learning E...	
10 Override difficulty #4	Todo	Iteration		Adaptive Learning E...	

+ You can use **Control + Space** to add an item

Figure 8: GitHub Projects Issues Overview

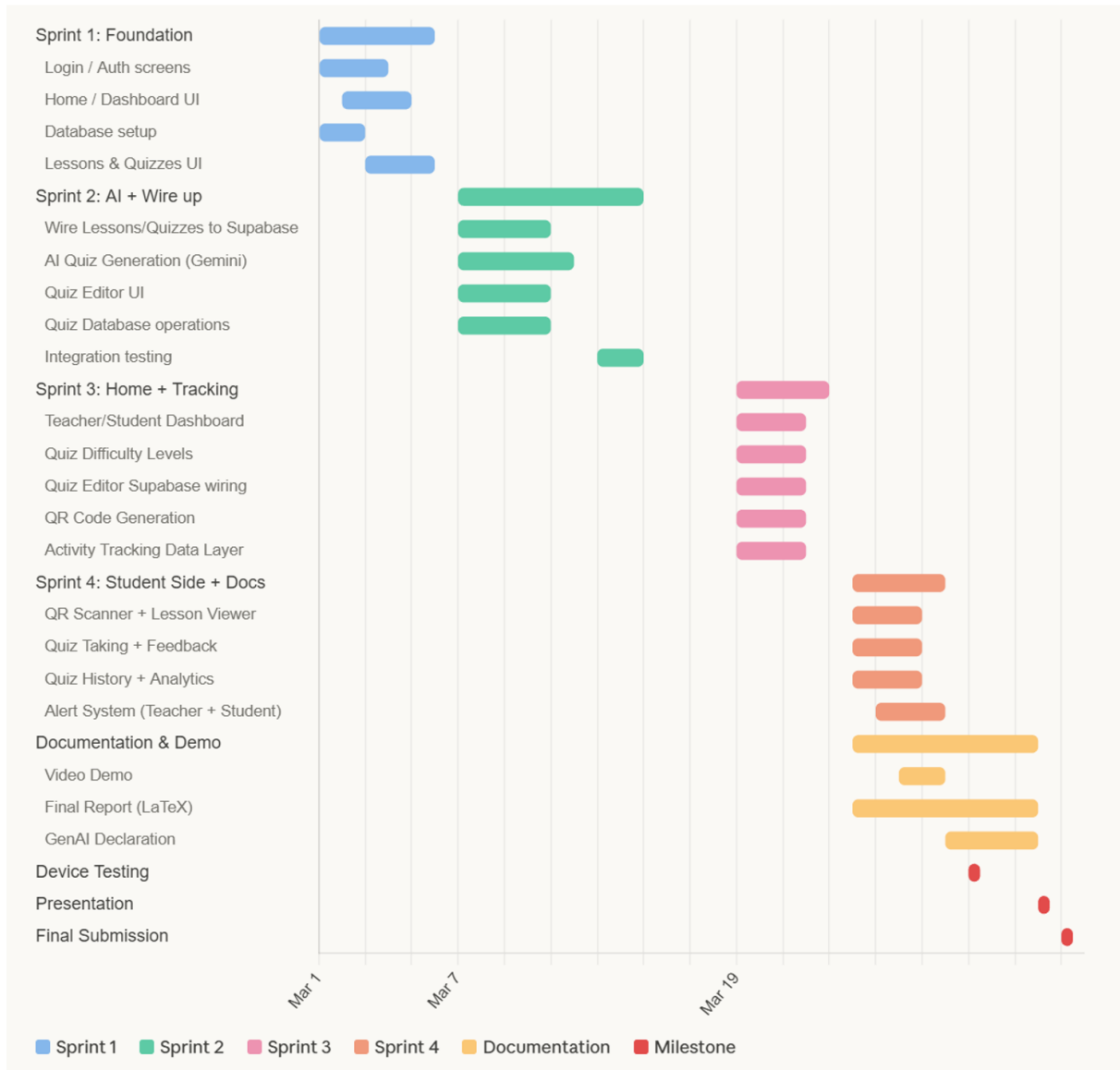


Figure 9: Gantt Chart

Appendix C: UI Implementation

Sign In

Email

Password

Login

Figure 10: Login Page



Figure 11: Teacher Home Screen

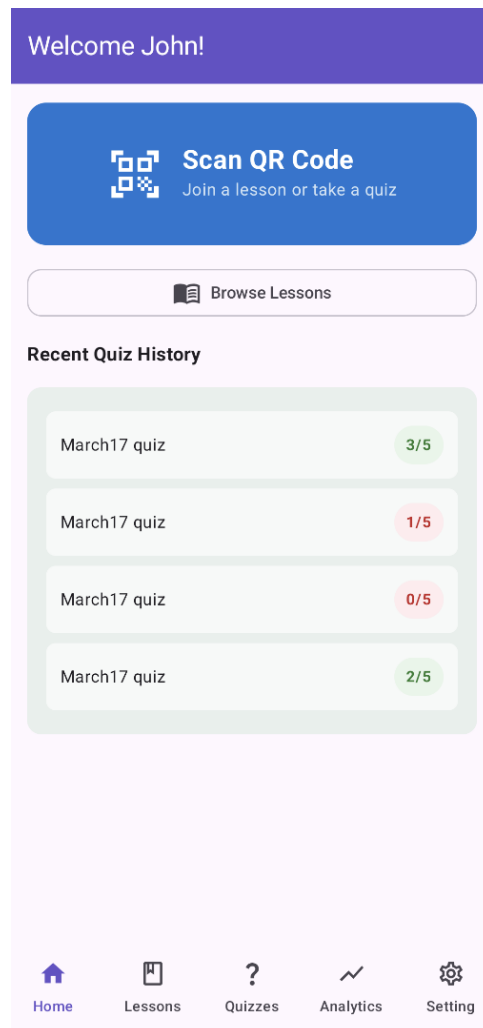


Figure 12: Student Home Screen

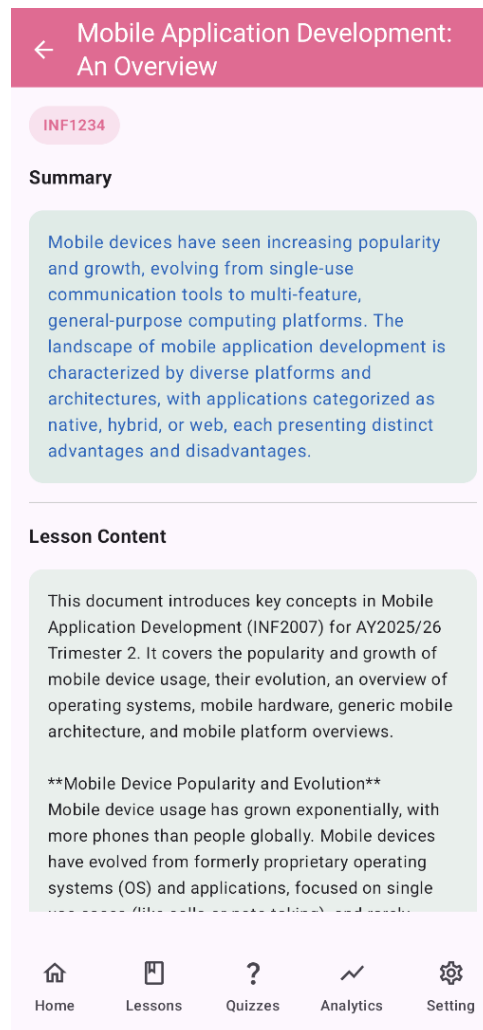


Figure 13: Lesson Content

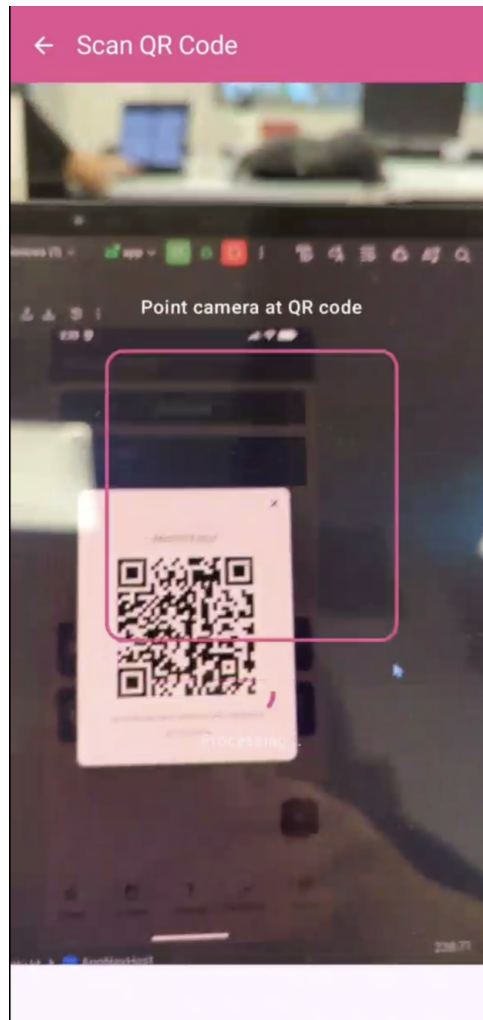


Figure 14: Student Scan QR Code

← Create Lesson

Build a lesson in minutes
Choose a module, upload a PDF, and let AI prepare the lesson content for you.

Lesson Details
Add the module and any optional context before generating. **3 modules**

<> Module Code *
Choose an existing module or type a new one

T Title (Optional)

Description (Optional)

Lesson File
Upload one PDF. AI will use it to create the lesson title, summary, and content.

📎

Figure 15: Teacher Create Lesson

← Create Quiz

Quiz Title

T Title (Optional)

Step 1: Select a Lesson

Select lesson

INF1234 - Mobile Application Dev

×

▼

✓

Mobile Application Development: An Overview

INF1234

Step 2: Number of Questions

5

10

15

20

Step 3: Difficulty

Difficulty

😊

Medium

▼

Ready to Generate

📖

Lesson: Mobile Application Development: An Overview

?

Questions: 5 MCQ

⚡

Difficulty: Medium

AI will use the lesson content to create the quiz.

Figure 16: Teacher Create Quiz

← Quiz for Kotlin Krash Kourse Save

Publish Quiz

Q1 ^ v [trash] ^

Question —

What is a key advantage of Kotlin concerning its relationship with Java?

Options — tap circle to mark correct answer

Option A —

☐ **A** Kotlin code compiles directly to Java

Option B —

☐ **B** Kotlin completely replaces Java

Option C —

☒ **C** Kotlin offers seamless interoperability with Java

Option D —

☐ **D** Kotlin is a dynamic language that runs on the JVM

✓ Correct: Option C – Kotlin offers seamless interoperability with Java, allowing Kotlin and Java classes to coexist in the same project.

Q2 ^ v [trash] ^

Figure 17: Teacher Edit Quiz

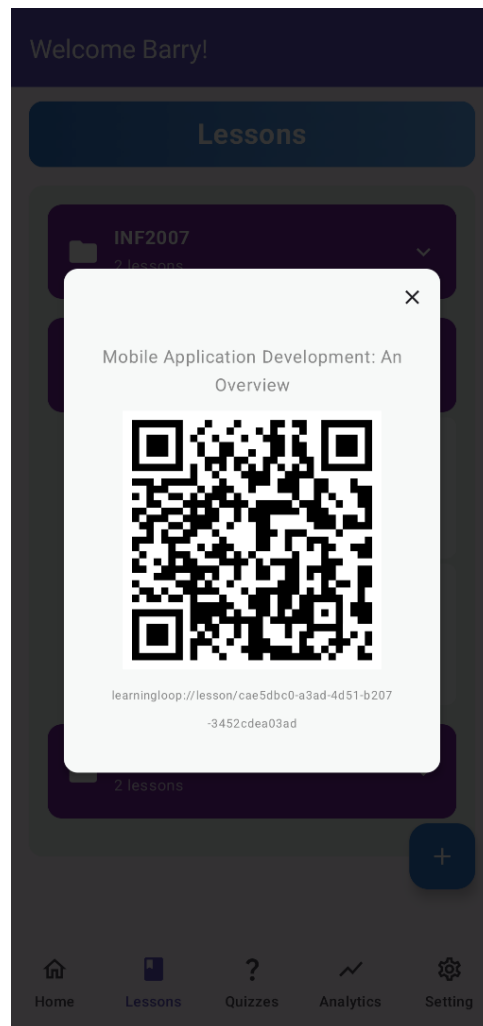


Figure 18: Teacher View QR Code

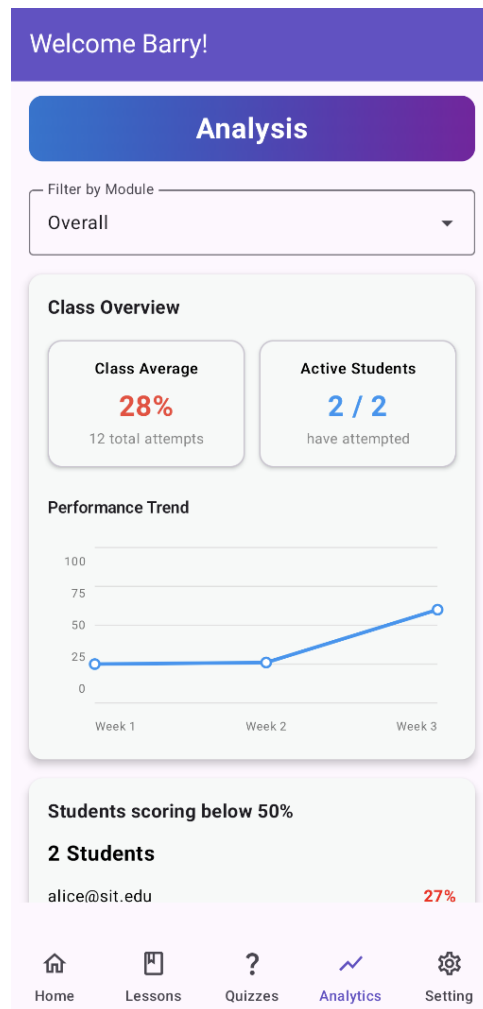
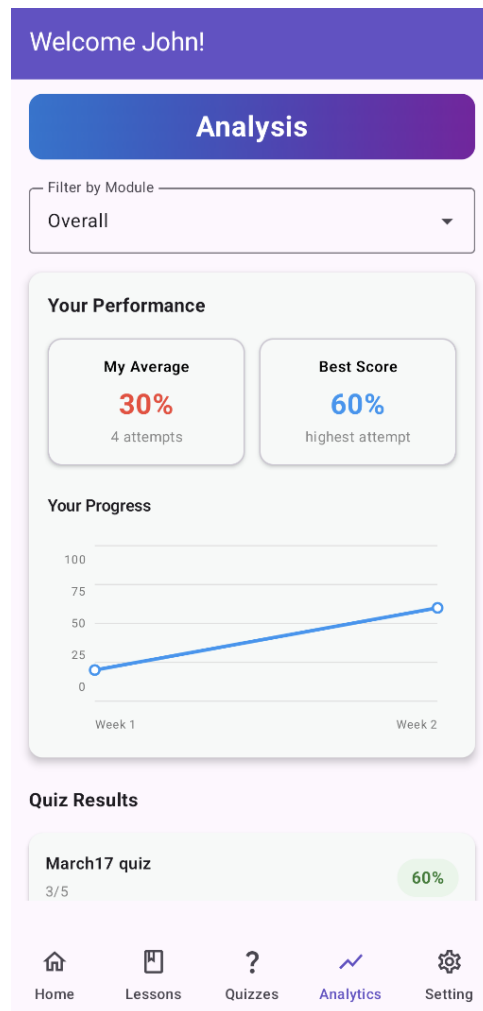


Figure 19: Teacher Analytics

**Figure 20:** Student Analytics

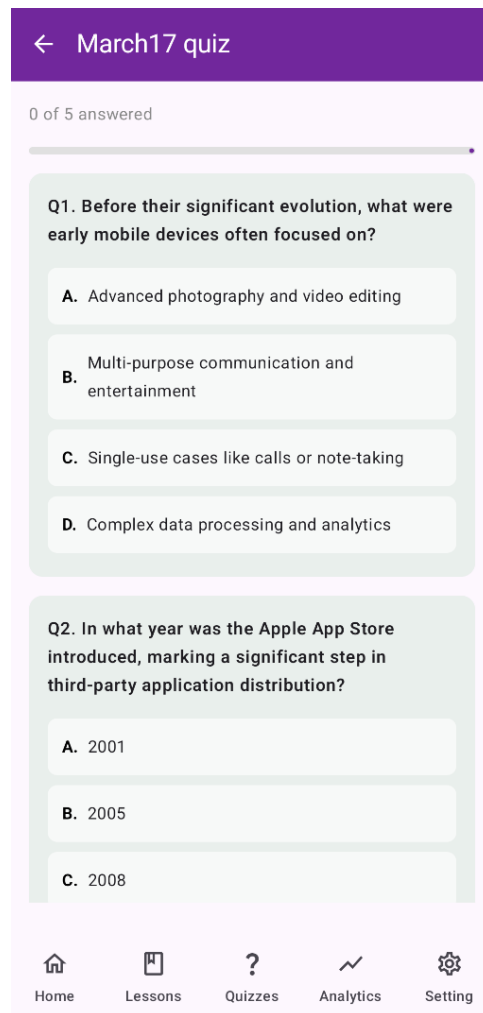
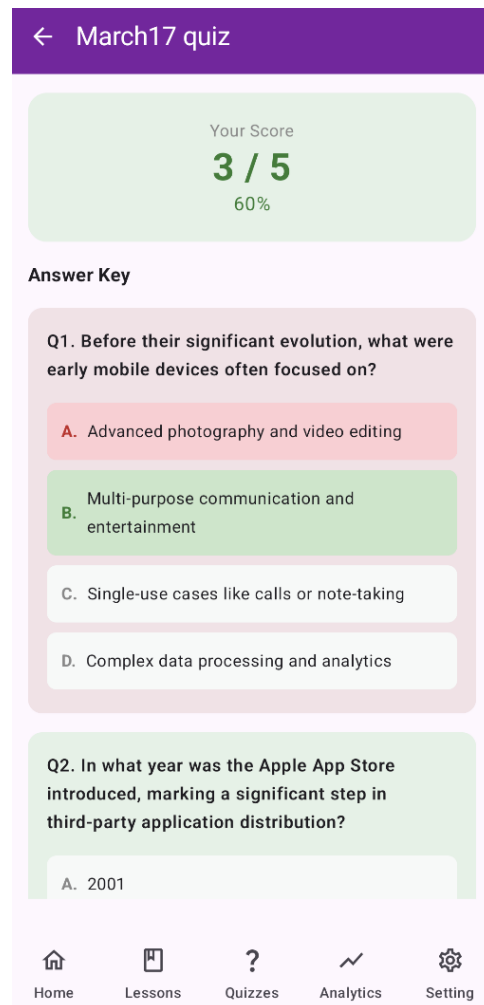


Figure 21: Student Quiz Taking

**Figure 22:** Student Quiz Review

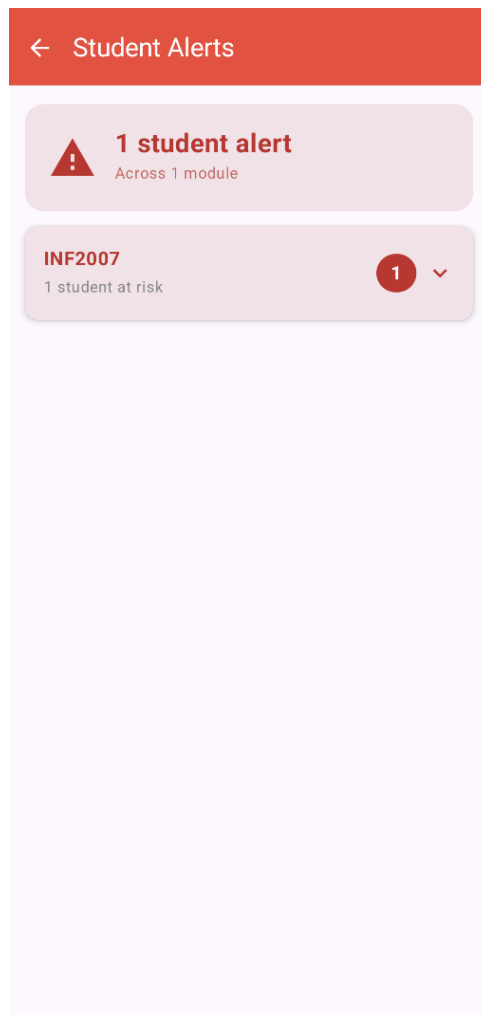


Figure 23: Teacher Alerts

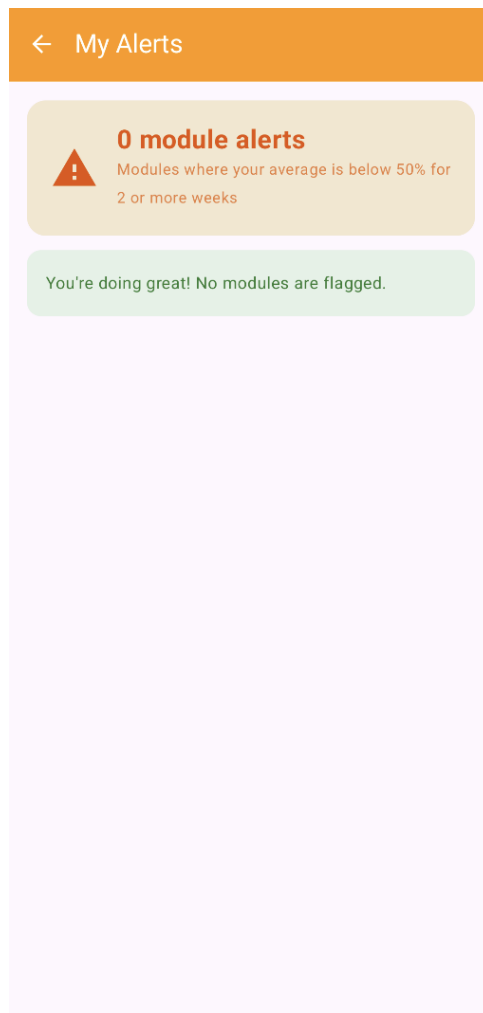


Figure 24: Student Alerts

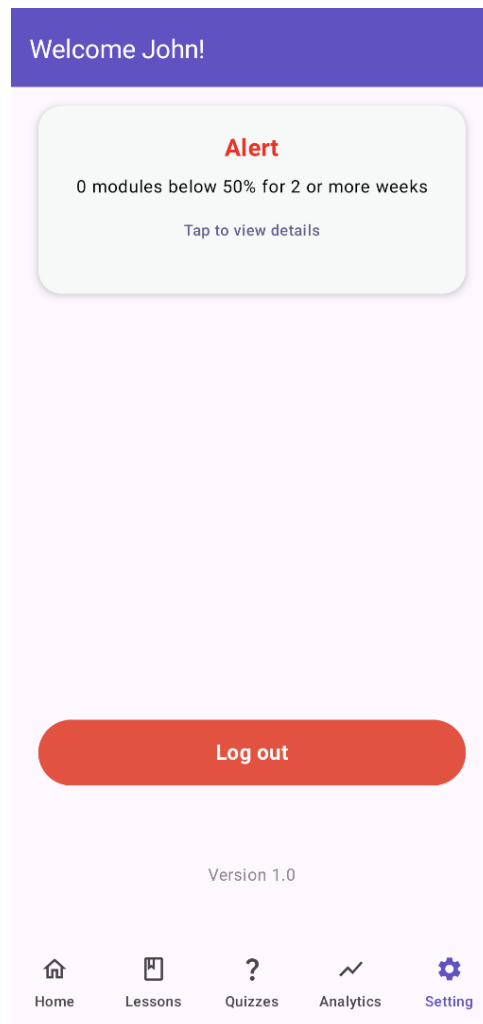


Figure 25: Settings Page

Table: Unit Test

Test File	Tests	Area Covered	What is Validated
QuizAttemptTest.kt	5	Score calculation	Average score computation from quiz attempts, including edge cases for empty attempts, single attempts, perfect scores, and zero scores.
StudentDeduplicationTest.kt	3	Student counting	Unique student de-duplication across modules, ensuring a student appearing in multiple modules is counted once in the overall view.
WeeklyTrendTest.kt	5	Performance trends	Weekly trend computation from attempt timestamps, including same-week averaging, five-week cap, null timestamp handling, and correct week labelling.
AnalyticFilterTest.kt	5	Module filtering	Module-based attempt filtering including overall mode, specific module filtering, unknown module handling, low scorer identification, and class average computation.
UserSessionTest.kt	6	Session management	UserSession singleton behaviour including field clearing on logout, correct role detection, default values when no user is set, and welcome text fallback for blank names.
ModuleGroupingTest.kt	6	Content organisation	Lesson grouping by module code, empty list handling, quiz filtering by publish status, quiz-to-module mapping, score percentage calculations, and quiz sorting by attempt count.
QuizAnswerTest.kt	6	Answer persistence	Answer correctness marking, saved answer mapping, empty answer list handling, attempt ID format validation, and correct versus wrong answer counting.

AlertTest.kt	14	Alert detection	Teacher alerts: empty attempts, students below 50% for two weeks flagged, students above 50% not flagged, single week not flagged, multiple students flagged separately, grouped by module, sorted by lowest average. Student alerts: module below 50% flagged, multiple quizzes combined per module, quiz details with individual scores, null timestamps skipped, sorted by lowest average.
LessonMappingTest.kt	1	AI content mapping	Validates that AI-generated quiz questions are correctly mapped from the Gemini API response format to the database row format, including question text, four options, and correct answer index.
ExampleUnitTest.kt	36	AI quiz parsing & quiz editor	Gemini API JSON parsing into GeminiQuizResponse and mapping to QuizQuestionRow. Quiz editor operations including updating, deleting, and validating question text. Quiz database model behaviour: default status, status toggling, and filtering by lesson or creator. Generation UI state management including <code>isGenerating</code> flag and error message capture.
AlertEdgeCaseTest.kt	8	Alert edge cases	Recovery behaviour when the most recent week is a pass (alert clears automatically). Boundary condition where exactly 50% is not treated as a failure. Graceful handling of unknown quiz IDs without crashing. Three consecutive failing weeks flagged with the correct count. Total attempt count accuracy per flagged module entry.

ConfidenceLevelTest.kt	8	Confidence level model	ConfidenceLevels.label() returns correct human-readable strings for NOT_CONFIDENT, OKAY, and VERY_CONFIDENT. null and unrecognised inputs return "Unknown". The all list contains exactly three items, and every item produces a non-blank, non-unknown label.
QuizStatusAndScoreTest.kt	14	Quiz status & score logic	Status toggle transitions between draft and published, including null status defaulting to draft. Score percentage computation for perfect, zero, half, fractional, and truncating cases, with division-by-zero protection. Pass/fail boundary at 50% (exactly 50% is a pass; 49% is a failure). QuizRow default status and field preservation on copy().
QuizViewModelLogicTest.kt	9	ViewModel logic	calculateAverageScore with mixed scores, differing total question counts, all-zero scores, and single perfect scores. Question reordering logic for first-to-last, last-to-first, and adjacent swap moves. updateQuestion replacing only the matching ID and leaving the list unchanged when the ID does not exist.
WeeklyTrendEdgeCaseTest.kt	8	Weekly trend edge cases	Single-attempt results for 100% and 0% scores return one entry. Two distinct weeks are ordered chronologically. Exactly five distinct weeks are all returned; six weeks are capped at five. Sequential "Week N" labels generated from index one. All-null timestamps return an empty result. Three attempts in the same week are averaged correctly to 80%.

Table 12: Unit Tests

Table: Unit Test Expected Result

Test Case	Input	Expected Output	Status
quizAttempt_calculatesAverageScore	Attempts: 8/10 and 6/10	Average = 0.7	Pass
quizAttempt_emptyAttemptsReturnsNull	Empty list	null	Pass
dedup_overallCountDeduplicatesStudentsAcrossModules	John and Alice each with 2 attempts across 2 modules	Unique students = 2, not 4	Pass
weeklyTrend_attemptsInSameWeekAreAveraged	Two attempts in same week: 80% and 60%	Single entry: Week 1 = 70%	Pass
weeklyTrend_returnsAtMostFiveWeeks	7 weeks of data	Result capped at 5 weeks	Pass
filter_specificModuleReturnsOnlyMatchingAttempts	Filter by INF2007 with mixed module attempts	Only INF2007 attempts returned	Pass
filter_lowScorersIdentifiesStudentsBelow50Percent	John avg 65%, Alice avg 25%	Alice flagged, John not flagged	Pass
userSession_clearsAllFields	Session with logged-in user, then clear()	isLoggedIn=false, email=empty, role=empty	Pass
quizAnswer_correctAnswerMarkedAsCorrect	Selected answer 2, correct answer 2	isCorrect=true	Pass
quizAnswer_wrongAnswerMarkedAsIncorrect	Selected answer 3, correct answer 2	isCorrect=false	Pass
failingStudents_studentBelow50ForTwoWeeksIsFlagged	Student scores 20% week 1 and 40% week 2	Student flagged with 2 failing weeks	Pass
failingStudents_studentAbove50IsNotFlagged	Student scores 80% week 1 and 100% week 2	Empty result, no flag	Pass
failingModules_combinesMultipleQuizzesInSameModule	4 attempts across 2 quizzes in INF2007	1 flagged module with 2 failed quizzes, 4 total attempts	Pass

aiQuiz_validJsonParsedInto GeminiQuizResponse	Valid Gemini JSON string	Parsed GeminiQuizResponse with correct question count	Pass
aiQuiz_emptyAiResponseThrowsException	Empty JSON string	Exception thrown	Pass
quizEditor_updateQuestionReplacesCorrectItemInList	List of 3 questions, update question at index 1	Only question 1 updated; others unchanged	Pass
quizEditor_blankQuestionTextIsInvalid	Blank question text string	Validation fails	Pass
quizDb_toggleStatusChange sDraftToPublished	QuizRow with status = "draft"	status = "published"	Pass
failingStudents_recoversAfterPass_notFlagged	Fail week 1, fail week 2, pass week 3	Result empty or weeksBelow50 < 2	Pass
failingStudents_exactly50Percent_notFlagged	Two weeks at exactly 50% (5/10)	Empty result, no flag	Pass
failingStudents_threeConsecutiveFailingWeeks_flaggedWithCorrectCount	20% score across 3 consecutive weeks	1 result with weeksBelow50 ≥ 3	Pass
label_null returnsUnknown	null passed to ConfidenceLevels.label()	"Unknown"	Pass
scorePercent_zeroTotal_doesNotDivideByZero	score = 5, total = 0	0	Pass
passFail_exactly50_isPass	score = 5, total = 10	pct < 50 is false	Pass
calculateAverage_allZeroScores	Two attempts both scoring 0/5	Average = 0.0	Pass
reorder_movesItemFromFirstToLast	List [Q1, Q2, Q3], move index 0 to end	[Q2, Q3, Q1]	Pass
weeklyTrend_twoDistinctWeeks_orderedChronologically	Earlier week 80%, later week 20%	Week 1 = 80%, Week 2 = 20%	Pass
weeklyTrend_allNullTimestamps_returnsEmpty	Two attempts with timestamp = null	Empty list	Pass

weeklyTrend_threeAttempts InSameWeek_averagedCorrectly	60%, 80%, 100% in same calendar week	1 entry = 80%	Pass
---	---	---------------	------

Table 13: Unit Test Expected Result**Table: Integration Tests**

Test File	Tests	Screen Tested	What is Validated
HomeScreenTest.kt	6	Home Screen	Teacher dashboard shows Upload Lesson and View Quizzes buttons but not Scan QR. Student dashboard shows Scan QR and Browse Lessons but not Upload Lesson. Welcome text displays username or role-based fallback. Teacher stat cards display Lessons, Quizzes, and Attempts labels.
AnalyticsScreenTest.kt	4	Analytics Screen	Teacher and student views both show the Analytics header and welcome text. Module filter dropdown is present. Welcome text falls back to role name when username is blank. PreferenceManager initialised in test setup to prevent crashes.
SettingsScreenTest.kt	5	Settings Screen	Logout button and Alert card are displayed. Version 1.0 text is shown. Logout and alert navigation callbacks trigger correctly on tap.
QuizReviewScreenTest.kt	5	Quiz Review Screen	Top bar with back button renders correctly. Back callback triggers on tap. Score percentage calculations for passing and failing scores are correct. Zero total questions handled without division error.
NavigationTest.kt	5	Navigation Flow	Upload Lesson, View Quizzes, Scan QR, and Browse Lessons buttons each trigger their respective navigation callbacks. UserSession clears all fields on logout.

LoginScreenTest.kt	6	Login Screen	“Sign In” heading, Email field, Password field, and Login button are all displayed. Email field accepts typed text and retains it on screen. Password field accepts input without crashing. PreferenceManager and UserSession initialised in test setup to prevent crashes.
LessonGenerationTest.kt	1	Lesson Generation Service	End-to-end generation chain executed: PDF copied from assets to cache, uploaded to Supabase Storage with text extracted, lesson generated and saved returning a non-null lesson ID, quiz generated from the lesson returning a non-null quiz ID. All three pipeline steps complete without throwing an exception.
StudentAlertScreenTest.kt	4	Student Alert Screen	“My Alerts” top bar title is displayed. Back button with content description “Back” exists. Back navigation callback triggers correctly on tap. Screen composes and reaches idle state without crashing.
TeacherAlertScreenTest.kt	4	Teacher Alert Screen	“Student Alerts” top bar title is displayed. Back button with content description “Back” exists. Back navigation callback triggers correctly on tap. Screen composes and reaches idle state without crashing.

Table 14: Integration Tests

Table: Integration Test Expected Result

Test Case	Setup	Action	Expected Output	Status
homeScreen_showsTeacherDashboard_whenRoleIsTeacher	UserSession.role = Teacher	Render Home-Screen	Upload Lesson visible, Scan QR not visible	Pass
homeScreen_showsStudentDashboard_whenRoleIsStudent	UserSession.role = Student	Render Home-Screen	Scan QR visible, Upload Lesson not visible	Pass
settingsScreen_logoutCallbackTriggered	Render SettingsScreen with callback	Tap Logout button	loggedOut = true	Pass
settingsScreen_alertNavigationCallbackTriggered	Render SettingsScreen with alert callback	Tap Alert card	alertClicked = true	Pass
homeScreen_scanQRCallbackTriggered_forStudent	UserSession.role = Student	Tap Scan QR Code card	navigated = true	Pass
loginScreen_showsSignInHeading	Render Login-Screen	—	“Sign In” text visible	Pass
loginScreen_emailFieldAcceptsInput	Render Login-Screen	Type test@sit.edu into Email field	“test@sit.edu” text visible	Pass
studentAlertScreen_backCallbackTriggered	UserSession.role = Student	Tap Back button	backPressed = true	Pass
teacherAlertScreen_backCallbackTriggered	UserSession.role = Teacher	Tap Back button	backPressed = true	Pass
teacherAlertScreen_rendersWithoutCrash	UserSession.role = Teacher	Wait for idle	“Student Alerts” visible, no crash	Pass

Table 15: Integration Test Expected Result

References

- [1] ABP Live. Creta class named best educational app for kids of the year 2022–2023 at indian education edtech awards, October 16 2023. pages 4
- [2] Duolingo, Inc. Duolingo surpasses 50 million daily active users, grows dau 36% and revenue 41 pages 5
- [3] S. Loewen, D. Crowther, D. R. Isbell, and et al. Mobile-assisted language learning: A duolingo case study. *ReCALL*, 31(3):293–311, 2019. pages 4
- [4] M. E. B. Santana, C. O. Jordão, V. B. dos Santos, and et al. Wing: An adaptive and gamified mobile learning platform for neurodivergent literacy. *arXiv*, 2025. pages 4